

DESARROLLO FULL STACK CON API REST, REACT Y REACT NATIVE

Objetivo

Desarrollar una solución para la gestión de superhéroes y misiones del universo Marvel, compuesta por un backend con API REST, una aplicación web desarrollada con React y una aplicación móvil desarrollada con React Native. El lenguaje, framework del backend y motor de base de datos son de libre elección.

Descripción del problema

Marvel necesita una plataforma sencilla para administrar información de superhéroes y sus misiones. El backend deberá centralizar la información y exponer una API REST protegida con JSON Web Token (JWT). La aplicación React permitirá administrar la información y la aplicación React Native permitirá consultarla desde un dispositivo móvil.

React y React Native deberán consumir la API desarrollada por el estudiante. No se aceptarán datos principales escritos manualmente en las interfaces.

Entidades

1. Usuario

- id
- nombre
- email
- password (encriptada)
- rol (ADMIN o CONSULTA)

2. Superhéroe

- id
- nombre
- nombre_real
- poder_principal
- nivel_poder (1 a 100)
- imagen_url
- estado (ACTIVO o INACTIVO)

3. Misión

- id
- titulo
- descripcion
- ubicacion
- fecha
- nivel_peligro (BAJO, MEDIO, ALTO)
- estado (PENDIENTE, EN_PROGRESO, COMPLETADA)
- superheroe_id

Endpoints requeridos

Autenticación

```
POST /api/auth/register
POST /api/auth/login
GET /api/auth/me
POST /api/auth/logout
```

Superhéroes

```
GET /api/heroes
GET /api/heroes/{id}
```



```
POST    /api/heroes
PUT      /api/heroes/{id}
DELETE  /api/heroes/{id}
```

Misiones

```
GET      /api/misiones
GET      /api/misiones/{id}
POST     /api/misiones
PUT      /api/misiones/{id}
DELETE   /api/misiones/{id}
```

Autenticación y autorización

- Implementar autenticación mediante JWT.
- Todos los endpoints, excepto Register y Login, deberán requerir un token válido.
- Rol ADMIN: acceso completo.
- Rol CONSULTA: únicamente consultas mediante GET.
- Si un usuario CONSULTA intenta crear, editar o eliminar información, responder HTTP 403.

Reglas de negocio

- No permitir usuarios con el mismo email.
- No permitir superhéroes con el mismo nombre.
- El nivel de poder debe estar entre 1 y 100.
- Una misión debe estar asociada a un superhéroe existente.
- La fecha de una misión es obligatoria.
- Solo se permitirán los estados y niveles de peligro definidos en este documento.

Validaciones mínimas

- Email válido y único.
- Password mínimo de 8 caracteres.
- Campos obligatorios validados.
- nivel_poder numérico entre 1 y 100.
- Identificadores inexistentes deberán responder HTTP 404.
- Los errores de validación deberán retornar una respuesta JSON comprensible.

Datos iniciales

El proyecto deberá incluir migraciones y/o script SQL y datos de prueba. Como mínimo:

- 2 usuarios: un ADMIN y un CONSULTA.
- 8 superhéroes inspirados en personajes de Marvel.
- 6 misiones asociadas a distintos superhéroes.

Aplicación web - React

La aplicación React deberá consumir la API desarrollada en el backend e implementar:

- Pantalla de Login y cierre de sesión.
- Rutas protegidas para usuarios autenticados.
- Listado de superhéroes mostrando imagen, nombre, poder, nivel y estado.
- Búsqueda de superhéroes por nombre.
- Pantalla de detalle de un superhéroe.
- Formulario para registrar y editar superhéroes.
- Eliminar un superhéroe solicitando confirmación.
- Listado de misiones y formulario para registrar o editar una misión.

Requerimientos técnicos de React

- Componentes funcionales y Props.
- useState y useEffect.
- React Router.
- Fetch o Axios para consumir la API.

- Formularios controlados.
- Renderizado condicional.
- Manejo de estados de carga, error y ausencia de datos.

Aplicación móvil - React Native

La aplicación móvil deberá consumir la misma API REST e implementar las siguientes pantallas:

- Login: autenticar al usuario y almacenar el JWT mediante AsyncStorage.
- Inicio: mostrar el nombre del usuario y accesos a Héroes, Misiones y Favoritos.
- Superhéroes: mostrar el listado obtenido desde la API utilizando FlatList.
- Detalle: mostrar imagen, nombre, nombre real, poder principal, nivel de poder y estado.
- Favoritos: permitir marcar héroes como favoritos y conservarlos mediante AsyncStorage.
- Misiones: mostrar el listado de misiones obtenido desde la API.

Requerimientos técnicos de React Native

- Componentes funcionales, Props, useState y useEffect.
- React Navigation.
- FlatList para los listados.
- Fetch o Axios para consumir la API.
- AsyncStorage para sesión y favoritos.
- Manejo de estados de carga y errores.

Consideraciones generales

- El backend, React y React Native deberán funcionar como una sola solución integrada.
- No se aceptarán arreglos de datos estáticos como reemplazo de la API.
- Se permite utilizar imágenes mediante URL.
- El estudiante deberá poder ejecutar y explicar las partes principales del proyecto.
- El código deberá estar organizado y evitar duplicaciones innecesarias.

Entregables

1. Código fuente completo del backend.
2. Script SQL o migraciones y datos iniciales.
3. Colección de Postman con pruebas de autenticación y CRUD.
4. Código fuente de la aplicación React.
5. Código fuente de la aplicación React Native.
6. Archivo README con instrucciones para instalar y ejecutar el proyecto.